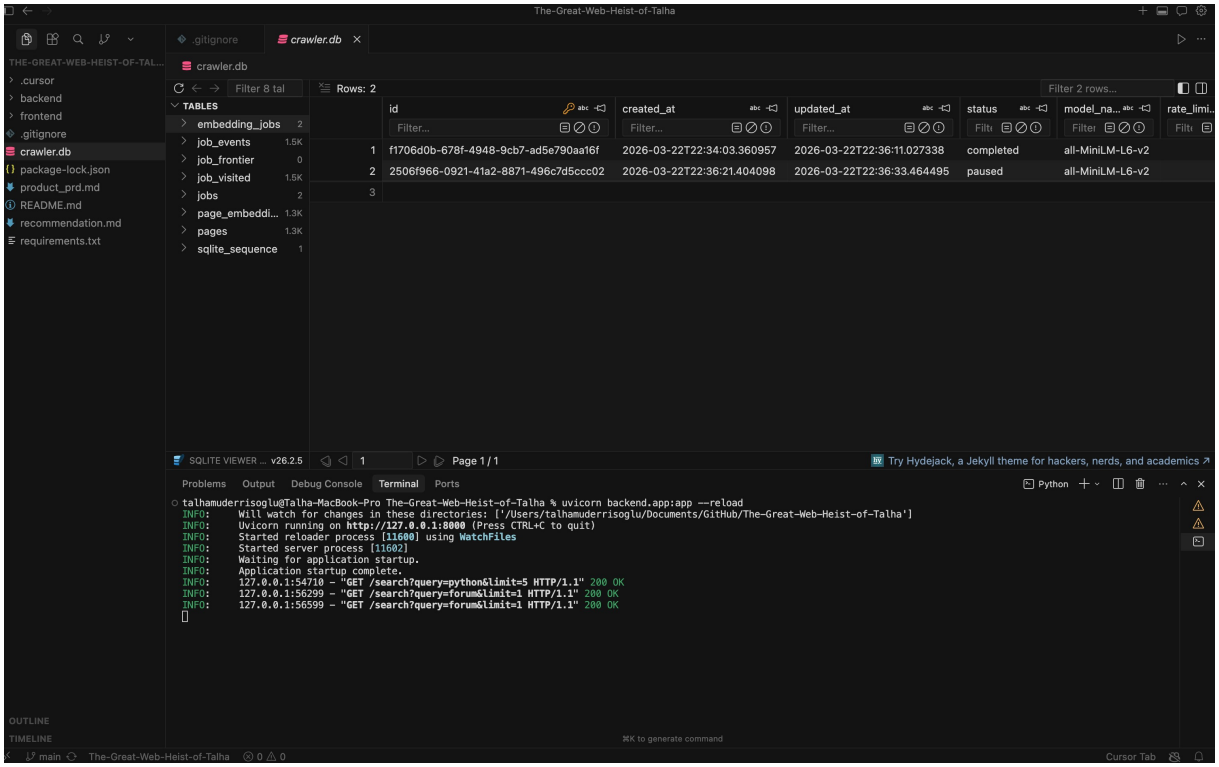
Talha Müderrisoğlu 820230335 Ai Aided
Computer Engineering Quiz 1 Answers.

1. Make sure you have crawled data available and also uploaded to GitHub. Open the raw storage file:



2. Find a word that appears on multiple different URLs (e.g., python, program, page). Write down the word you chose: forum

3. For that word, copy 3 entries from the file (each line is: word url origin depth frequency):

   (format: word url origin depth frequency)

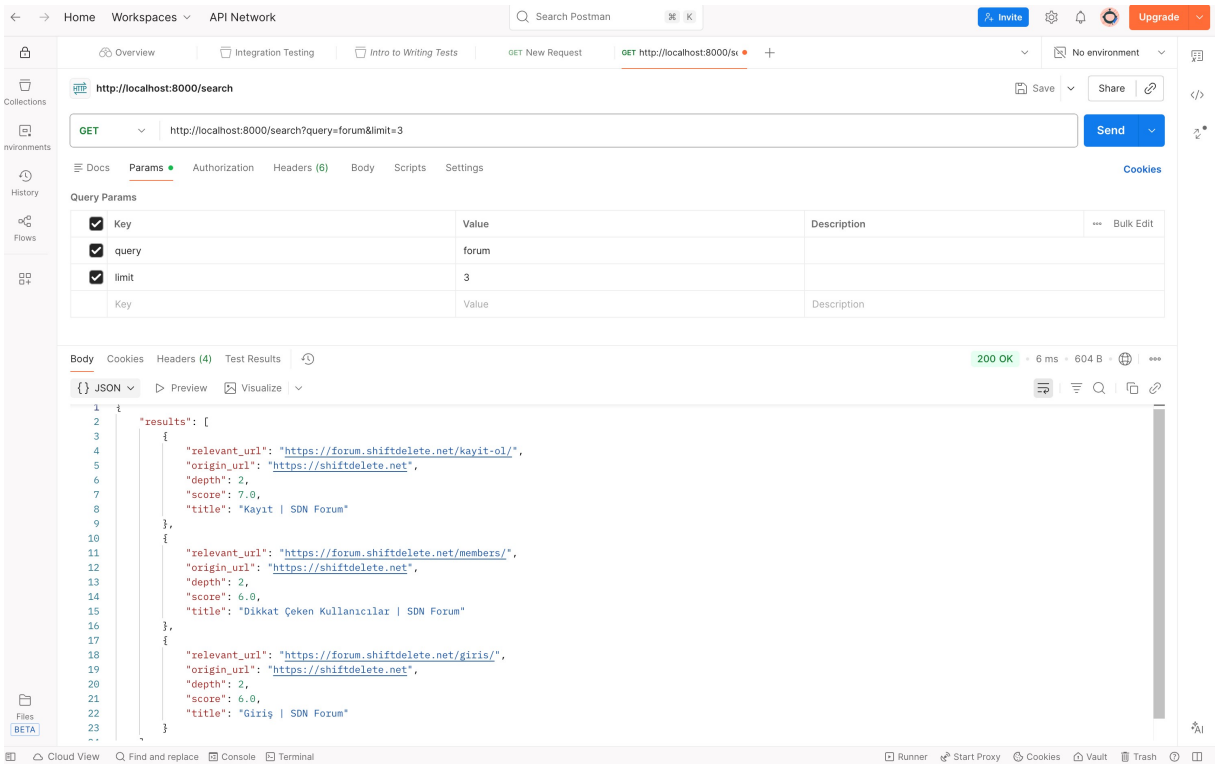
Entry 1: https://forum.shiftdelete.net/kayit-ol/ https://shiftdelete.net 2 7

Entry 2: https://forum.shiftdelete.net/forumlar/akilli-telefonlar.452/ https://shiftdelete.net 2 6

Entry 3: https://forum.shiftdelete.net/forumlar/aliexpress.489/ https://shiftdelete.net 2 6

4. Now search for that word via the API:

   GET http://localhost:8000/search?query=forum&limit=3



I already store my relevance scores from api call sorted. So no need for sorting in the API Call:

```python
    def search(self, query: str, limit: int | None = None) -> List[Tuple[str, str, int, float, str]]:
        tokens = tokenize(query)
        if not tokens:
            return []

        scores: DefaultDict[str, float] = defaultdict(float)
        meta: Dict[str, Tuple[str, int, str]] = {}

        for token in tokens:
            for url, origin_url, depth, score in self.inverted.get(token, []):
                scores[url] += score
                if url not in meta:
                    title = self.pages.get(url).title if url in self.pages else ""
                    meta[url] = (origin_url, depth, title)

        ranked = sorted(scores.items(), key=lambda item: item[1], reverse=True)
        if limit is not None:
            ranked = ranked[:limit]
        results: List[Tuple[str, str, int, float, str]] = []
        for url, score in ranked:
            origin_url, depth, title = meta.get(url, ("", 0, ""))
            results.append((url, origin_url, depth, score, title))
        return results
```

5. Write down the #1 result's URL and relevance_score:

- URL: `https://forum.shiftdelete.net/kayit-ol/`

- relevance_score: 7.0

6. Now manually calculate the score for each of your 3 entries using the formula:

score = (frequency x 10) + 1000 (exact match bonus) - (depth x 5)

- Entry 1 score: (__7_ x 10) + 1000 - (__2_ x 5) = _____1060____
- Entry 2 score: (__6_ x 10) + 1000 - (__2_ x 5) = _____1050___ - Entry 3 score: (__6_ x 10) + 1000 - (__2_ x 5) = _____1050___

7. Does the highest score you calculated match the API's #1 result? Yes / No: Yes

8. How could you enhance the process in a Chain-of-Thought Manner. Explain.

Use a structured mul--step ranking pipeline: normalize query (lowercase, tokenize, remove noise), retrieve candidate URLs from inverted index, compute per-candidate sub-scores (term frequency, depth penalty, exact-match bonus, -tle bonus), combine into final relevance score with weights, sort and return top-N with an explana-on object (score breakdown), evaluate with test queries and tune weights.

Add -e-breakers: fresher pages, shorter URL paths, domain priority.

Keep an offline evalua-on set (query -> expected top URLs) to measure improvements consistently.